

Web Services Tutorial



Tamara Kogan
tkogan@cincom.com

About this tutorial



- **Purpose of this tutorial:**
 - provide an introduction to Web services technology
 - display VW's support of Web services technology
 - explain the use of Web services tools available in VW

Introduction to Web Services



Web Services Technology

- What are Web services about?
- The Web Services Model
- Enabling Technologies
- SOAP messages
- Web Services Description Language

What are Web Services about

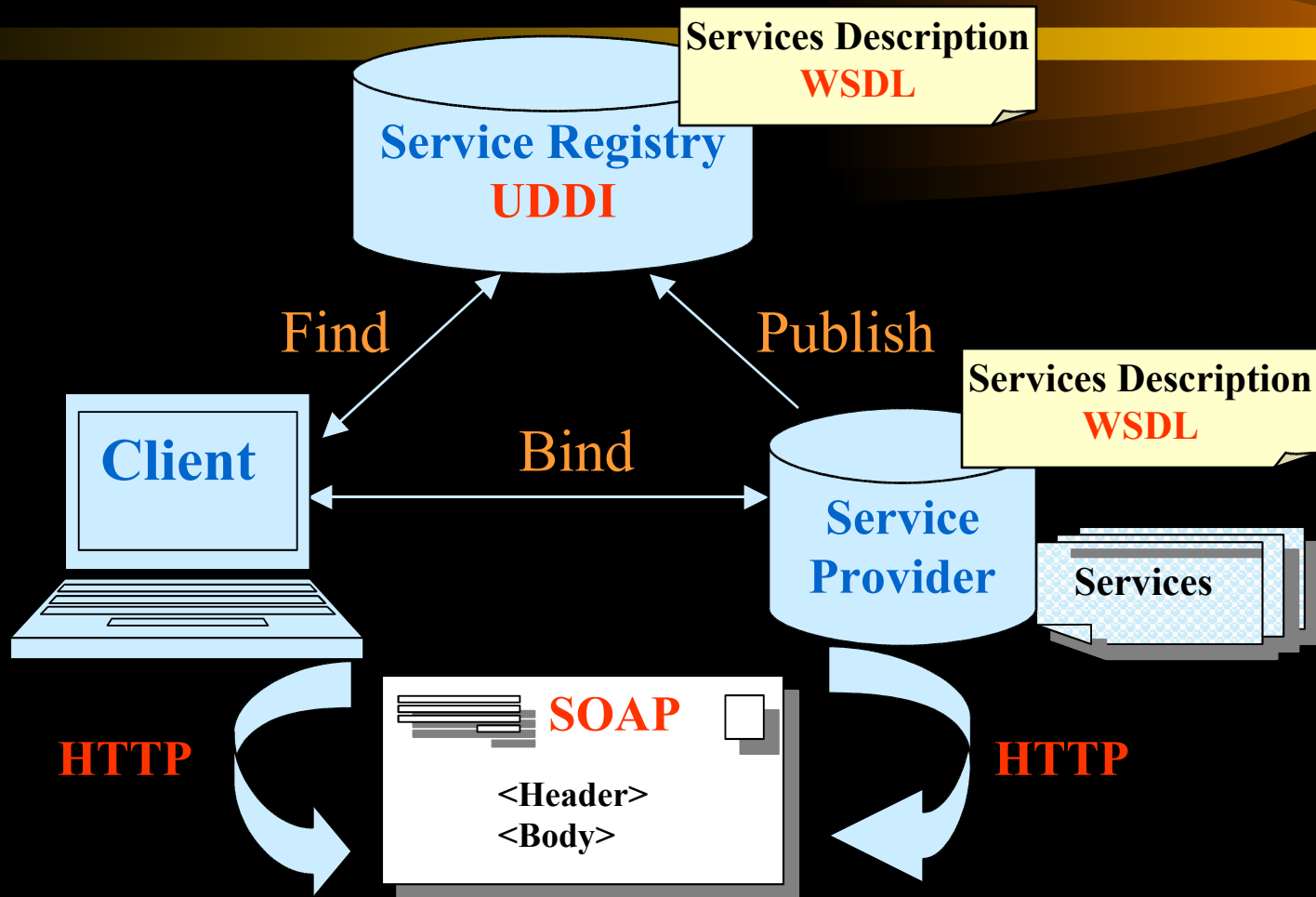
- **Web Services**

is a technology that allows applications to communicate with each other in a platform- and programming language-independent manner.

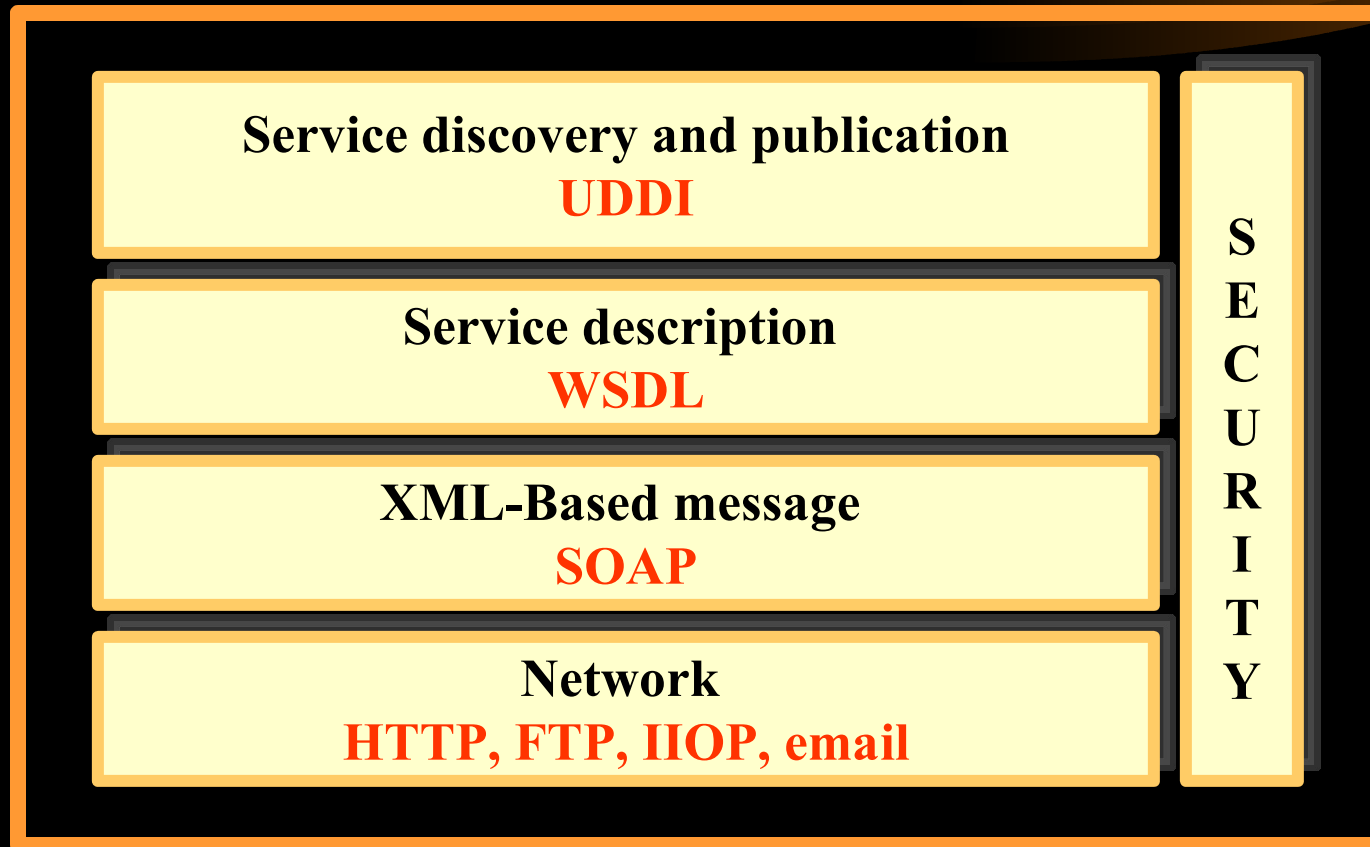
- **A Web Service**

is a software interface that describes a collection of operations that can be accessed over the network through standardized XML messaging. It uses protocols based on the XML language to describe an operation to execute or data to exchange with another Web service.

The Web Services Model



Enabling Technologies



SOAP 1.1 Message

<SOAP-ENV:Envelope

xmlns:SOAP-ENV="http://schemas.xmlsoap.org/soap/envelope/">

<SOAP-ENV:Header>

<m:Transaction

SOAP-ENV:actor="http://schemas.xmlsoap.org/soap/actor/next"

SOAP-ENV:mustUnderstand="1">5

</m:Transaction>

</SOAP-ENV:Header>

<SOAP-ENV:Body>

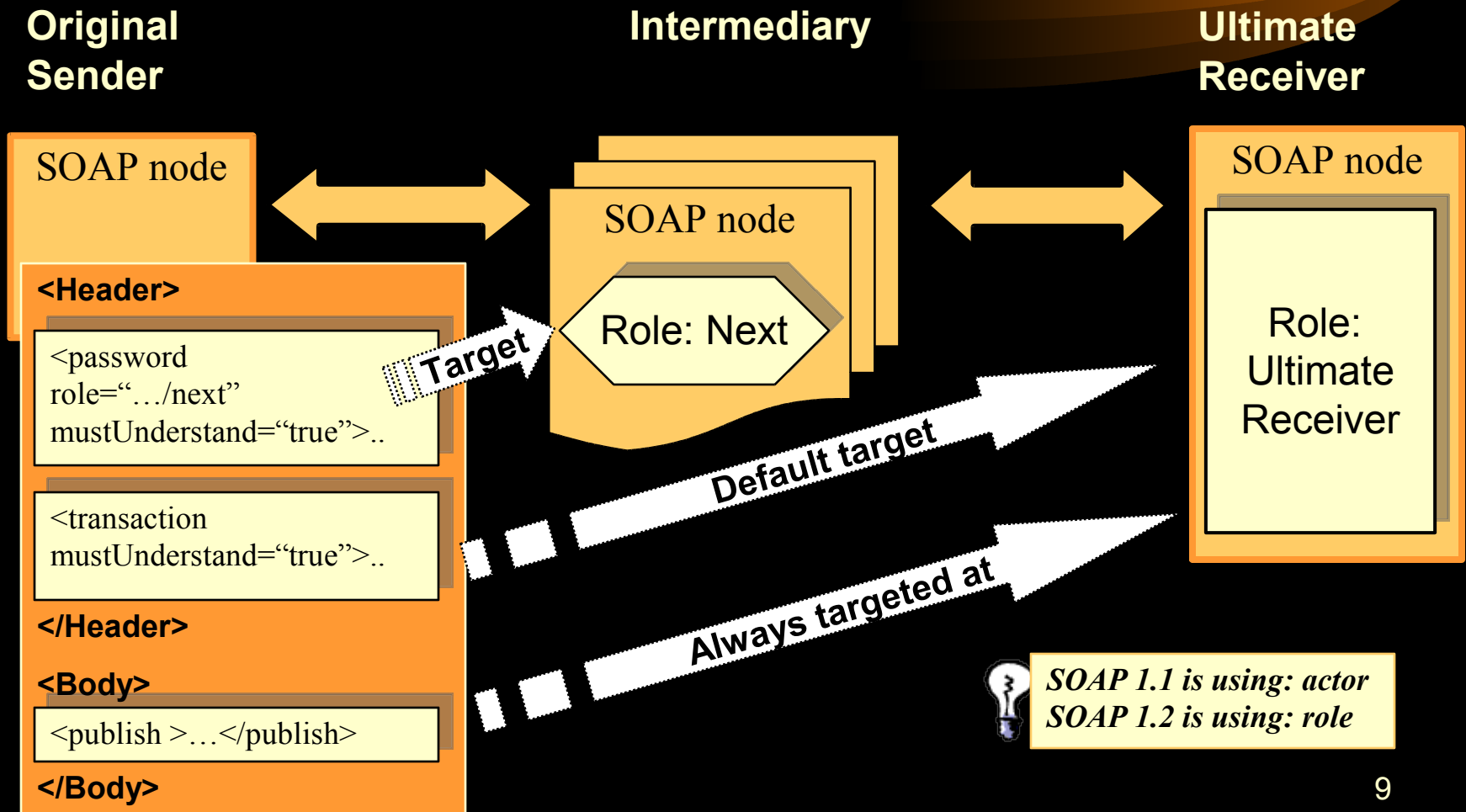
<m:HoldingByAcquisitionNumber xmlns:m="Some-URI">1234567

</m:HoldingByAcquisitionNumber>

</SOAP-ENV:Body>

</SOAP-ENV:Envelope>

SOAP Message Transmission



Soap 1.1 Message over HTTP

POST /LibrarySearch HTTP/1.1
Host: www.libraryserver.com
Content-Type: text/xml; charset="utf-8"
Content-Length: nnnn
SOAPAction: "Some-URI"

```
<SOAP-ENV:Envelope  
  <SOAP-ENV:Header>  
    ...  
  </SOAP-ENV:Header>  
  <SOAP-ENV:Body>  
    ....  
  </SOAP-ENV:Body>  
</SOAP-ENV:Envelope>
```

SOAP Fault Message

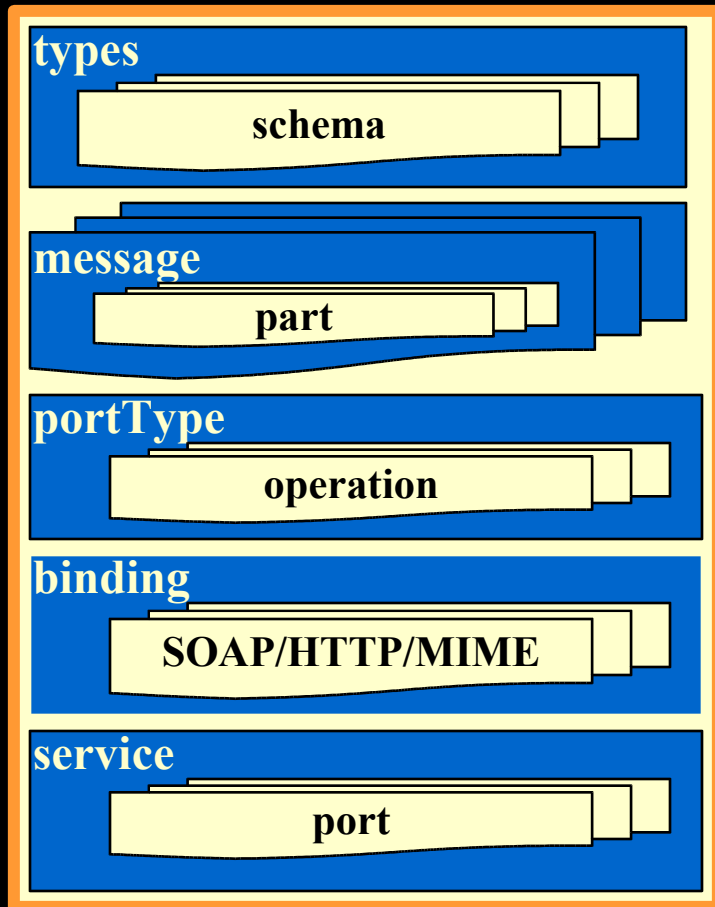
HTTP/1.1 500 Internal Server Error

Content-Type: text/xml; charset="utf-8"

Content-Length: nnnn

```
<SOAP-ENV:Envelope.. >
  <SOAP-ENV:Body>
    <SOAP-ENV:Fault>
      <faultcode>SOAP-ENV:Server</faultcode>
      <faultstring>Server Error</faultstring>
      <detail>
        <e:myfaultdetails xmlns:e="Some-URI">
          <message>My application didn't work</message>
          <errorcode>1001</errorcode>
        </e:myfaultdetails>
      </detail>
    </SOAP-ENV:Fault>
  </SOAP-ENV:Body>
</SOAP-ENV:Envelope>
```

WSDL Schema



Data types

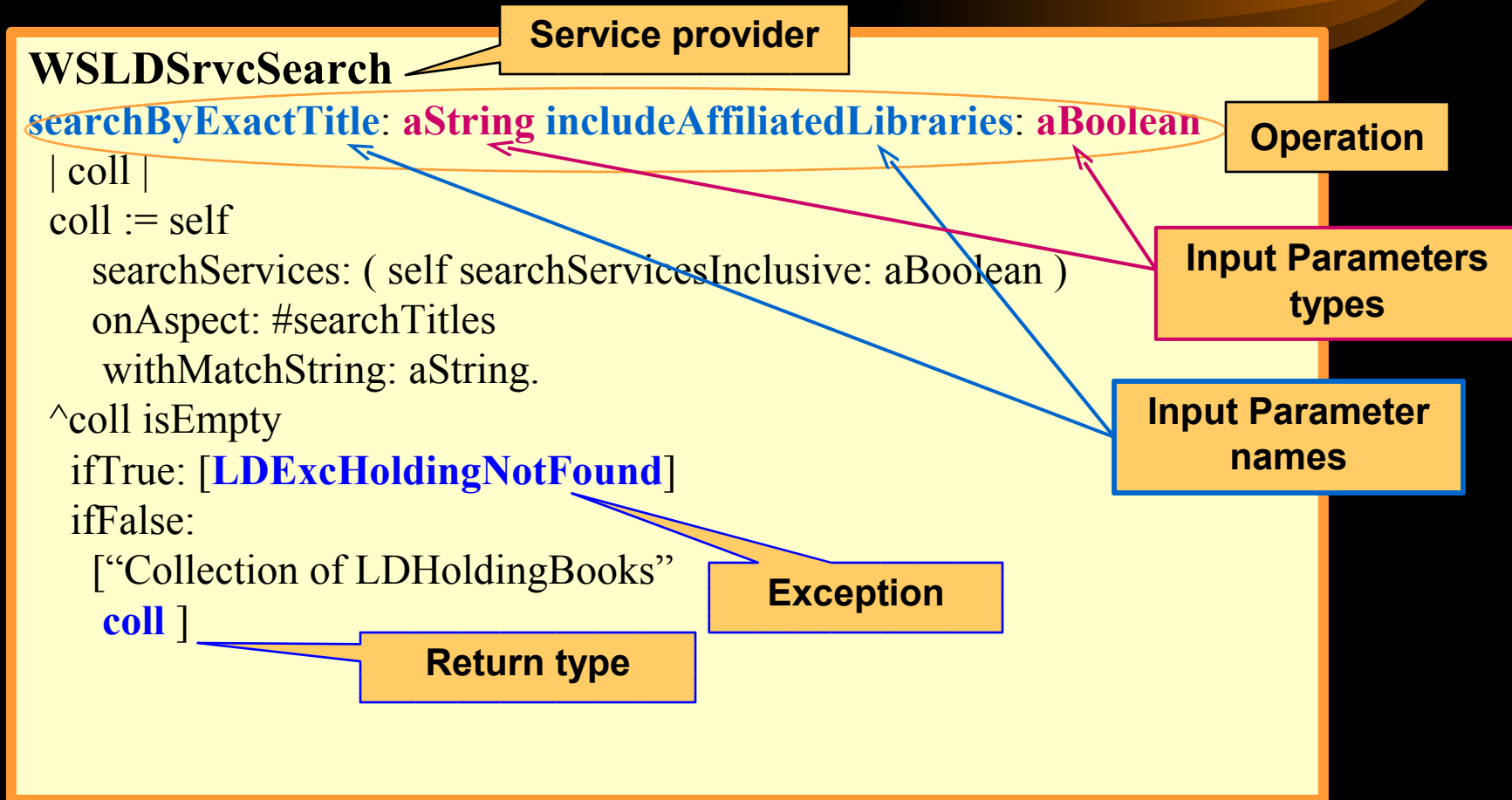
Operation parameter types

Operation input and output parameters

Package details based on specific protocol

Service location and binding

Describing a service in WSDL



Describing parameter types

WSLDSrvcSearch

searchByExactTitle: aString

includingAffiliatedLibraries: aBoolean

RPC style:

```
<message name="SearchByExactTitleSoapIn">
  <part name="SearchByExactTitle" type="xsd:string"/>
  <part name="IncludeAffiliatedLibraries" type="xsd:boolean"/>
</message>
<message name="SearchByExactTitleSoapOut">
  <part name="return" type="ns:CollectionOfLDHoldingBook"/>
</message>
```

Document style:

```
<message name="SearchByExactTitleIncludeAffiliatedLibrariesSoapIn">
  <part name="parameter" element="ns:SearchByExactTitleIncludeAffiliatedLibraries"/>
</message>
<message name="SearchByExactTitleIncludeAffiliatedLibrariesSoapOut">
  <part name="return" element="ns:SearchByExactTitleIncludeAffiliatedLibrariesResponse"/>
</message>
```

WSDL RPC and Document styles

- Document/literal
 - Message has one or zero parts
 - Part is resolved using an element
 - The element is complex type in most cases
 - Data is serialized according to a schema
- RPC/encoded
 - The Soap body contains an element with the name of a remote procedure
 - Message can have zero or more parts
 - Each part corresponds a remote procedure parameter
 - Each part is resolved using type
 - Data is serialized according to SOAP 1.1

Describing types

```
<wsdl:types>
  <wsdl:schema targetNamespace="urn:someURL">
    <complexType name="LDHoldingBook">
      <sequence>
        <element name="dueDate" type="xsd:date"/>
        <element name="language" type="xsd:string"/>
        ....
      </sequence>
    </complexType>
    ....
  </wsdl:schema>
</wsdl:types>
```

Document style describing
parameter types

```
<element
  name="SearchByExactTitleIncludeAffiliatedLibraries">
  <complexType>
    <sequence>
      <element name="searchByExactTitle" type="xsd:string"/>
      <element name="includeAffiliatedLibraries"
        type="xsd:boolean"/>
    </sequence>
  </complexType>
</element>
```

Describing interfaces

WSLDSrvcSearch

searchByExactTitle: aString

includingAffiliatedLibraries: aBoolean

```
<portType name="WSLDSrvcSearch">
  <operation name="SearchByExactTitleIncludeAffiliatedLibraries">
    <input message="ns:SearchByExactTitleSoapIn"/>
    <output message="ns:SearchByExactTitleSoapOut"/>
  </operation>
  <operation ...>
    ....
  </operation>
  .....
</portType>
```

Describing message transfer

```
<binding name="WSLDSrvSearch"
  type="ns:WSLDSrvSearch">
  <soap:binding style="document"
    transport="http://schemas.xmlsoap.org/soap/http/">
    <operation name="SearchByExactTitleIncludeAffiliatedLibraries"
      selector="searchByExactTitle:includeAffiliatedLibraries:">
      <soap:operation soapAction="">
        <input>
          <soap:body use="literal" namespace="urn:Library\doc"/>
        </input>
        <output>
          <soap:body use="literal" namespace="urn:Library\doc"/>
        </output>
        <fault name="LDExcHoldingNotFound" .. />
      </operation>
    </binding>
```

Transport SOAP
over HTTP

VW specific, not
spec complaint

Not used by VW

Message
input/output wire
presentation

Exception
description

Describing service location

```
<service name="LibraryServices">  
  <port name="LibrarySearch" binding="ns: WSLDSrvcSearch ">  
    <soap:address location="http://localhost:3933/searchRpc"/>  
  </port>  
</service>
```

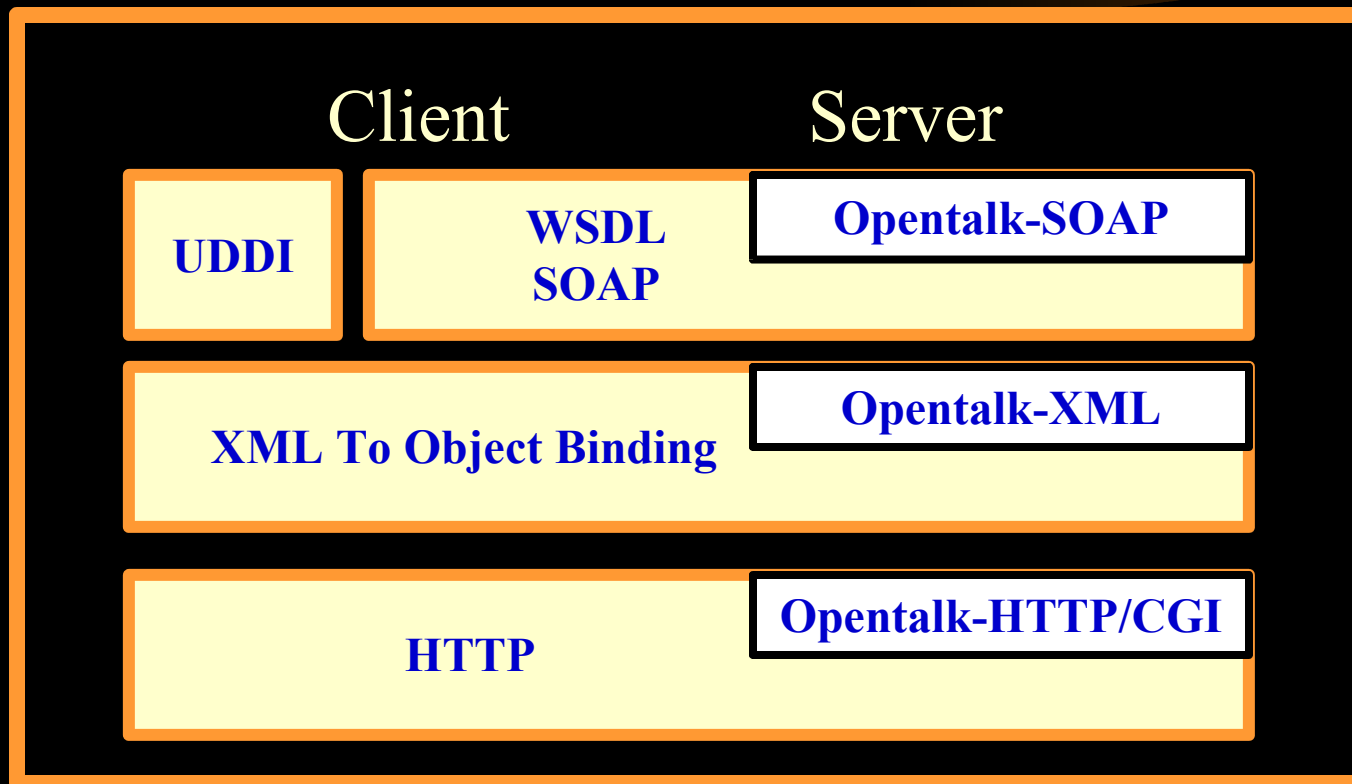
Access point for
SOAP binding

Can have one or more ports, each of which define a
connection method (for example, HTTP/SMTP, etc)

Web Services Support in VW



Web Services Frameworks



Currently Supported Protocols



- SOAP 1.1
 - Released SOAP 1.2 spec
- WSDL 1.1
 - WSDL 1.2 spec work in progress
- UDDI v1
 - Released UDDI v2 and v3 spec's

VW Web Services Toolkit

- **Provides support in**
 - **creating XML to object binding from a schema**
 - **marshaling/unmarshaling XML types in to Smalltalk object and visa versa**
 - **creating classes from XML to object binding**
 - **building and deploying Web services from an existing application**
 - **creating classes from a WSDL schema and accessing Web services**
 - **searching and publishing Web services in UDDI registry**

Web Services Invocation



Loading WSDL Schema

- WsdlBinding
 - loads and register a WSDL schema
 - creates Wsdl configuration
 - serves as a repository of WSDL schemas

WsdlBinding

```
loadWsdlBindingFrom: self wsdlSpecification  
readStream
```

How to access Web Services

- WsdlClient
 - quick and easy way to invoke a service
 - doesn't create any classes
- WsdlClassBuilder
 - create client classes to invoke a service
 - can create Opentalk server and client and service classes
- WsdlWizard
 - GUI tool available in vw 73

WsdClient

- Loads and parses a Wsd schema
- Creates default binding to dictionaries
- Invokes a Web services

```
client := WsdClient
```

```
url: 'http://www.xmethods.net/sd/2001/CurrencyExchangeService.wsdl'.
```

```
rate := client
```

```
    executeSelector: #getRate
```

```
    args: (Array with: 'usa' with: 'canada')
```

WsdClassBuilder

- Loads and parses a Wsd schema
- Creates XML to object binding and classes from it
- Creates client, server and service classes

```
builder := WsdClassBuilder readFrom:  
'http://www.xmethods.net/sd/2001/CurrencyExchangeService.wsdl' asURI.  
builder package: 'CurrencyExchange'.
```

Creating WSDL Client Class

clientClass := builder createClientClasses first.

- derived from WsdIClient
- created for each WSDL port

```
Smalltalk defineClass: #CurrencyExchangePortClient  
    superclass: #{WebServices.WsdIClient}
```

```
#getRateCountry1: aString country2: aString1
```

```
#initialize
```

```
    super initialize.
```

```
    self setPortNamed: 'CurrencyExchangePort'.
```

```
#class wsdISchema
```

```
    "(WebServices.WsdIBinding loadWsdIBindingFrom:  
        self wsdISchema readStream.)" ...
```

Port is registered in
WsdIPort.PortRegistry

WSDL schema with
XML to object binding

Testing WSDL Client

```
client := clientClass new.  
client createScript inspect.  
rate := client  
    getRateCountry1: 'usa'  
    country2: 'canada'
```

Creating Service Class Stub

serviceClass := builder createServiceClasses first.

```
Smalltalk defineClass: #CurrencyExchangeBinding  
    superclass: #{Core.Object}
```

```
getRateCountry1: aString country2: aString1  
<operationName: #getRate >  
<addParameter: #country1 type: #String >  
<addParameter: #country2 type: #String >  
<result: #Float >
```

```
^self "Add implementation here"
```

Creating Opentalk Client

clientClass := builder createOpentalkClientClasses first.

```
Smalltalk defineClass: #OpentalkClientCurrencyExchangePort
```

```
  superclass: #{Core.Object}
```

```
  instanceVariableNames: 'client proxy '
```

Remote Object

```
#getRateCountry1: aString country2: aString1
```

```
  ^proxy getRateCountry1: aString country2: aString1
```

Request Broker

```
#serverUrl
```

```
  ^'http://services.xmethods.net:80/soap'
```

WSDL schema with XML
to Object Binding

```
#class wsdSchema
```

```
  "(WebServices.WsdBinding
```

```
    loadWsdBindingFrom: self wsdSchema readStream.)"
```

```
.
```

Creating Opentalk Server

```
builder opentalkServerName: 'ExchangeServer'.  
serverClass := builder createOpentalkServerClass.
```

```
Smalltalk defineClass: #ExchangeServer
```

```
instanceVariableNames: 'interfaces servers'
```

Request
Brokers

```
#portDescription
```

```
<serviceClass: #CurrencyExchangeBinding
```

```
address: #'http://services.xmethods.net:80/soap'
```

```
bindingType: #soap
```

```
wSDLBinding: #CurrencyExchangeBinding
```

```
<wSDLServiceImplementation: #CurrencyExchangeService >
```

```
^self
```

Corresponds
WSDL <port>
element

Testing locally

- ✓ Implement service method:

```
serviceClass>>getRateCountry1:country2:  
^123
```

- ✓ Change server port to a local host:

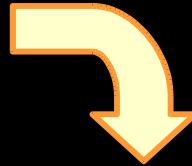
```
serverClass class>>portDescription  
<serviceClass: #'WebServices.LibraryServices' address:  
  #'http://localhost:4920' ... >
```

- ✓ Set server access point for the client

```
clientClass>>serverUrl  
^'http://localhost:4920'
```

Testing Opentalk Server and Client

```
server := serverClass new.  
server startServers.  
  “Invoke client request”  
server stopServers.
```



```
client := clientClass new start.  
[client  
  getRateCountry1: 'usa'  
  country2: 'canada'  
] ensure: [ client stop ]
```

WsdlClassBuilder Settings

- Default package
 - WSDDefaultPackage
- Default proxy client port
 - 4930
- Use existing classes or generate a new uniquely named class
 - yes

WSDL Wizard

Released in 7.3

Showing

Show Time Review



- Loaded a Wsdl schema
- Created XML to object binding
- Created classes from the binding
- Created a client for each port
- Created a script to invoke services

Building and Deploying Web Services



Steps to build Web Services

- Provide services description
 - Provide description to service interfaces
 - Provide description to service parameters, result and exception types
- Create a Wsdl schema
- Create Opentalk server
- Create Opentalk client

Classes to do the job

- WsdlBuilder
 - expects service and types description
 - creates a WSDL schema from a service class
- WsdlClassBuilder
 - creates Opentalk server and client classes
- WSDLWizard
 - helps to describe types
 - creates Opentalk server and client classes
 - tests client-server communication
 - creates Wsdl schema

Service description

Should include:

- Operation name
- Parameter , result and exception types

WSLDSrvGeneralPublic

holdingByAcquisitionNumber: anAcquisitionNumber

<operationName: #'HoldingByAcquisitionNumber'>

<addParameter: #'acquisitionNumber' **type:** #'LargePositiveInteger'>

<result: #'LDHoldingBook'>

<addException: #NotFound **type:** #'LDExcHoldingNotFound'>

^library ownedHoldings

detect:[:x | x acquisitionNumber = aLDHolding_acquisitionNumber]

ifNone:[LDExcHoldingNotFound raise]

Types description

Currently supported pragma types:

- Simple types
- Complex types
- Collections
- Choice
- Soap Array
- Struct

LDAgent

#borrowedHoldings: aCollOfLDHoldingBook

<**addAttribute**: #(#borrowedHoldings #optional)

type: #(#Collection #"WebServices.LDHoldingBook)>

borrowedHoldings := aCollOfLDHoldingBook

To be resolved should
be fully qualified

Creating a WSDL Schema

```
builder := WsdlBuilder  
  buildFromService: WSLDSrvGeneralPublicDoc.
```

```
builder  
  setPortAddress: 'http://localhost:5050/srvGeneralDoc'  
  forBindingNamed: 'WSLDSrvGeneralPublicDoc'  
  wsdlServiceNamed: 'LibraryDemoSoapDoc'.
```

```
stream := String new writeStream.  
builder printSpecWithSmalltalkBindingOn: stream.
```

WsdBuilder Settings

- Default target namespace
 - the same target namespace is used for a WSDL schema definition and types element
- Add the service super class methods
- Add selector attribute
- Style and use attributes
 - Document/RPC encoded/literal
- Default service protocol
 - the methods from this protocol are used to create Wsd1 operations
- Default class namespace
 - is used in XML to object binding to resolve types

WSDL Wizard

Showing

Show Time Review

- Described service parameters, result and exception types
- Described data types
- Created Opentalk server
- Created Opentalk client
- Tested client server communication
- Created a Wsdl schema

Interoperability

- Document/literal schema style
 - WS-I recommended
 - Default in .NET
- Problem with RPC/encoded
- Inline type

```
xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance"
<item xsi:type="xsd:string">abc</item>
```
- Object identity

```
<inputStructArray href="#id3"/>
<struct SOAP-ENC:arrayType="ns1:SOAPStruct[3]" id="id3">
```

...Interoperability

- Support for XML “anyType”
 - <element name=“reference”/> - default type: “anyType”
 - Simple types
 - Complex types
- Support for nil attribute
 - Schema description
 - <element name="varInt" type="long" nillable="true"/>
 - <element name="varInt" type="long"/> - nillable=“false“
 - Message encoding
 - <varStruct xsi:nil="true"/>

Data Serialization Settings

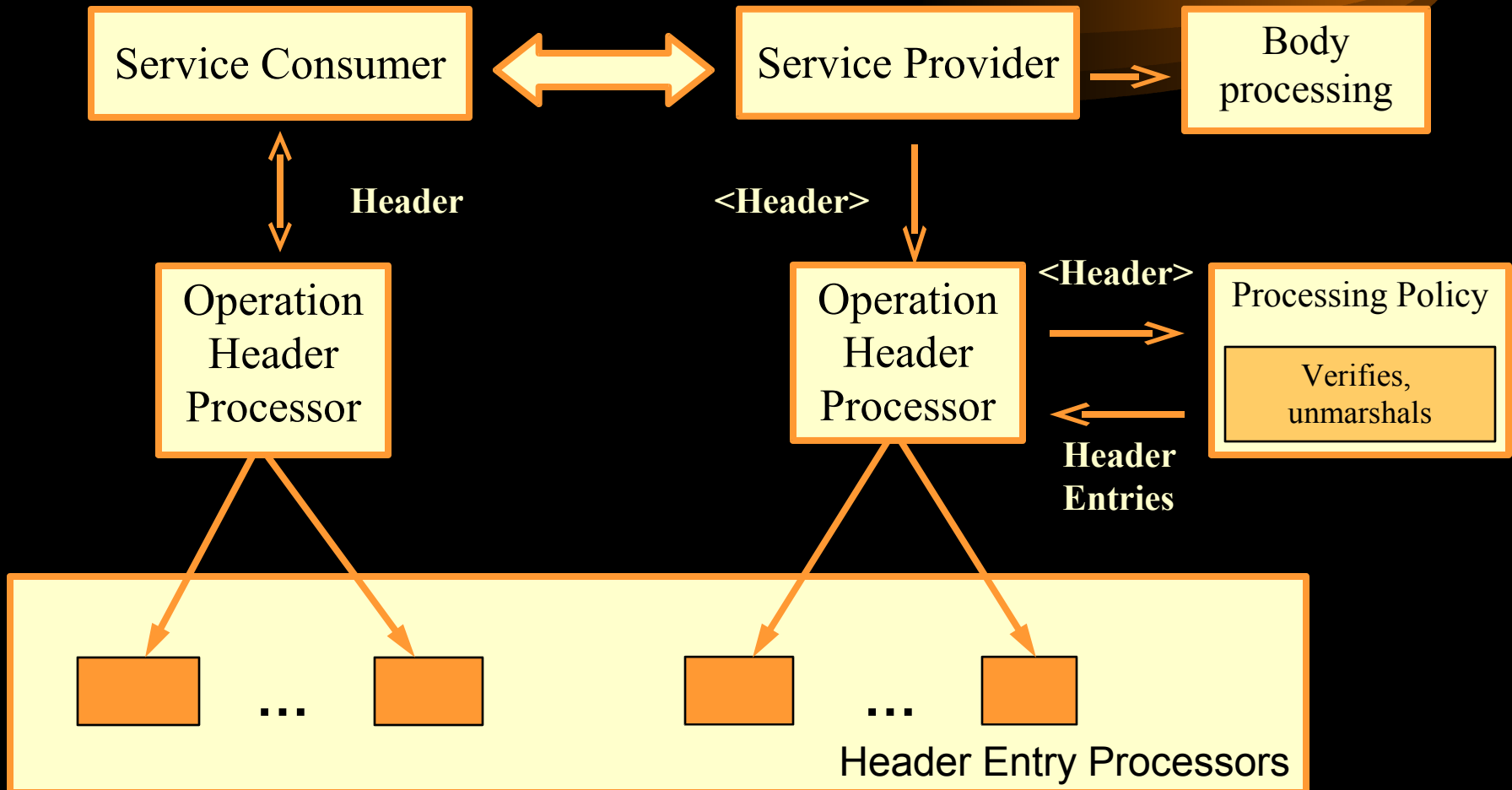
- XMLObjectMarshalingManager
 - #nillableDefault
 - #useInlineType
 - #useNilAttribute
 - #useReference
- SoapArrayMarshaler
 - #useEmptyLengthForDimension

<...SOAP-ENC:arrayType="xsd:string[]"“../>

SOAP Header Support

- Wsdl client support in 7.2
 - Add, marshal and unmarshal header entry
 - No verification
- SOAP header processing model
 - Preview for 7.3
 - Opentalk client and server support
 - Opentalk-SOAP-HeadersDemo package

SOAP Headers Processing Model



Sending SOAP Headers

```
Opentalk.SOAPMarshaler defaultReturnedObject: #envelope.  
client := Smalltalk.CustomerClient new.  
client start.  
(client headerFor: #AuthenticationToken)  
    value: ( AuthenticationToken new  
                userID: 'UserID';  
                password: 'password';  
                yourself).  
envelope := client setCustomerID: 1234.  
headerStruct := envelope header.  
(headerStruct at: #Confirmation) value return = 'confirmed'  
    ifFalse: [ self error: 'wrong result'].
```

Opentalk Client Settings

There are a few options to set the Opentalk client result in SOAPMarshaler defaultReturnedObject

- #result – returns the body value, default
- #envelope - returns instance of WebServices.SoapEnvelope, having an envelope as a result allows to get access to response header and body
- #response - returns a SoapResponse, the result can be helpful for debugging purpose

Tutorial Wrap-up

- In this tutorial, we've done the following things:
 - Learned about Web Services technology.
 - Learned about how to describe an interface using WSDL schema
 - Reviewed VW Web Services Tool.
 - Used the WS Tool to create Web service based systems from WSDL files.
 - Created and deployed a Web service system from an existing application
 - Learned about SOAP header processing model

Resources

- XML
 - <http://www.w3.org/TR/xmlschema-2/>
- SOAP 1.1 specification
 - <http://www.w3.org/TR/soap/>
- WSDL 1.1 specification
 - <http://www.w3.org/TR/wsdl.html>
- UDDI specification
 - http://www.oasis-open.org/committees/tc_home.php?wg_abbrev=uddi-spec
- WS-I basic profile
 - <http://www.ws-i.org/Profiles/BasicProfile-1.0-2004-04-16.html>

